

Instituto Tecnológico y de Estudios Superiores de Monterrey.

Escuela de Ingenierías

**Maestría en Inteligencia Artificial Aplicada – Proyecto Integrador Grupo 10,
Equipo 47.**



Profesores:

Dra. Grettel Barceló Alonso
Dr. Luis Eduardo Falcón Morales
Mtra. Verónica Sandra Guzmán de Valle
Dr. Gerardo Jesús Camacho González
Dr. Eusebio Vargas Estrada

Modelo Final

Equipo #47

Erick Eduardo Betancourt Del Angel	A01795545
Lucero Guadalupe Contreras Hernández	A01794502
Erik Morales Hinojosa	A01795110

Fecha de entrega: 2 de Noviembre del 2025

Repositorio de GitHub:

<https://github.com/erikmoralestec/proyecto-integrador-grafos-de-conocimiento-llm>

I. Situación actual

Durante los avances previos (documentados en los Notebooks 8, 9 y 10), se logró una integración funcional entre tres componentes esenciales de la arquitectura de recuperación aumentada propuesta: (i) la recuperación semántica a través de Qdrant como almacén vectorial, (ii) la consulta estructurada mediante el grafo de conocimiento en Neo4j, y (iii) la generación de respuestas por medio de modelos de lenguaje de gran escala (LLMs). Esta integración permitió construir un flujo híbrido de tipo vector-grafo-LLM, capaz de responder consultas complejas con razonamiento contextual y trazabilidad documental.

No obstante, dicha arquitectura aún operaba bajo un enfoque monolítico y secuencial, donde las etapas de clasificación, recuperación, razonamiento y síntesis se ejecutaban de forma lineal y sin validación cruzada. Esta estructura, aunque funcional, presentaba limitaciones relevantes en términos de escalabilidad, adaptabilidad ante ambigüedad semántica, y capacidad para integrar múltiples fuentes de evidencia de manera convergente. En particular, se observó que ante preguntas con múltiples entidades, ambigüedad ontológica o requerimientos de inferencia estructural, el sistema tendía a generar respuestas incompletas o excesivamente genéricas.

A partir de estas observaciones, el presente avance se orientó a una transformación sustantiva del núcleo del sistema: se migró hacia una arquitectura de tipo **multiagente coordinada**, sustentada en **LangGraph** como motor de orquestación. Esta arquitectura se basa en la implementación de un grafo computacional donde cada nodo corresponde a un agente especializado que ejecuta una función concreta dentro del flujo de resolución de consultas: clasificación de intención, recuperación semántica, traversal de grafos, síntesis argumentativa y validación crítica, entre otras.

El uso de LangGraph permitió diseñar un flujo asíncrono, ramificado y validable, en el que los agentes colaboran de manera coordinada pero autónoma, compartiendo un contexto común actualizado dinámicamente. De esta manera, el sistema deja de ser una secuencia de pasos rígidos y se convierte en una red adaptativa de razonamiento distribuido, con mayor capacidad para procesar preguntas complejas, establecer vínculos entre evidencias heterogéneas y proporcionar respuestas justificadas con trazabilidad explícita.

Este avance —documentado principalmente en el **Notebook 11: MultiAgent-GraphRAG**— representa un punto de inflexión en el desarrollo del sistema, y se estructura en los siguientes ejes analíticos:

- **(II)** Diseño e implementación del sistema multiagente, especificando la lógica de orquestación, definición de agentes funcionales, estructura del grafo y mecanismos de coordinación asíncrona.
- **(III)** Obstáculos conceptuales y técnicos derivados de la coordinación multiagente, como la gestión de contexto compartido, ambigüedad en la clasificación de intención, y sincronización de evidencias parciales.
- **(IV)** Comparación con la arquitectura monolítica previa (Notebook 10), destacando las ventajas observadas en modularidad, escalabilidad, mantenibilidad y trazabilidad de la inferencia.

- (V) Proyecciones hacia una arquitectura robusta de inferencia asistida, con agentes adicionales orientados a control de confianza, generación de explicaciones estructuradas, y facilitación de revisión humana en casos límite.

En suma, la transición hacia una arquitectura multiagente habilita una nueva etapa en el desarrollo del sistema, orientada no solo a la mejora del rendimiento operativo, sino también a la consolidación de una plataforma explicativa, auditable y extensible, capaz de integrar fuentes vectoriales, estructurales y lingüísticas en un marco de razonamiento coherente y verificable

II. Arquitectura Multiagente con LangGraph

La adopción de una arquitectura multiagente representó un cambio paradigmático en la forma en que el sistema aborda la resolución de consultas complejas. Mientras que en versiones anteriores las operaciones se ejecutaban de forma secuencial bajo un esquema monolítico, el nuevo enfoque permitió descomponer el flujo de razonamiento en múltiples agentes especializados, cada uno con una función claramente definida, operando dentro de un grafo computacional orquestado mediante LangGraph.

LangGraph, como extensión de LangChain, permite modelar flujos conversacionales como máquinas de estado dinámicas, donde cada estado representa un agente con una tarea específica y cada transición está condicionada por el estado del contexto compartido (state object). Esta abstracción habilitó una mayor modularidad, control sobre los puntos de decisión, y capacidad para realizar ciclos de validación y retroalimentación dentro del flujo.

2.1 Agentes funcionales del grafo

Se diseñó un conjunto de agentes con funciones complementarias que, al operar en conjunto, permiten capturar la intención del usuario, recuperar evidencia contextual, acceder a conocimiento estructurado, y generar respuestas argumentadas y verificables. La Tabla 1 resume los agentes implementados, sus responsabilidades y los insumos/salidas esperados en cada caso.

Agente	Descripción funcional	Entrada esperada	Salida generada
Specialized Router	Detecta qué tipos de entidades requiere la pregunta (Recall, Complaint, Investigation) mediante análisis de palabras clave y contexto semántico. Activa	Texto de la pregunta en lenguaje natural.	Lista de agentes activos (<code>active_agents`</code> ), tipo de query ( <code>query_type`</code>), y contexto inicial.

	agentes especializados correspondientes.		
RecallAgent	Agente especializado exclusivamente en queries sobre nodos `Recall` en Neo4j. Genera y ejecuta queries Cypher específicos para campañas de retiro, validando que el query contenga la entidad correcta.	Pregunta del usuario, estado compartido.	Resultados de Recall almacenados en `recall_results`, IDs de recalls en `shared_context`.
ComplaintAgent	Agente especializado exclusivamente en queries sobre nodos `Complaint`. Genera queries Cypher para quejas de consumidores, validando presencia de `COMPLAINT` en el query generado.	Pregunta del usuario, estado compartido.	Resultados de Complaint en `complaint_results`, IDs de complaints en `shared_context`.
InvestigationAgent	Agente especializado en queries sobre nodos `Investigation`. Genera queries con validación de dirección correcta de relación `RELATES_TO` (Investigation → Recall).	Pregunta del usuario, estado compartido.	Resultados de Investigation en `investigation_results`, IDs de investigations en `shared_context`.
GraphAgent	Agente general para queries de grafo cuando no hay agentes especializados activos. Genera queries Cypher genéricos para análisis estructural del grafo.	Pregunta del usuario, entidades detectadas.	Subgrafo relevante con nodos, relaciones y metadatos asociados.
RelationshipCreatorAgent	Identifica relaciones faltantes entre entidades descubiertas por los agentes especializados. Propone queries de creación usando `MERGE` para evitar duplicados, con métricas de calidad.	Resultados de agentes especializados, `shared_context`.	Query Cypher de creación de relaciones (`relationship_query`) con score de calidad (0-8.0).
ConfirmationAgent	Valida y confirma queries de creación de relaciones antes de ejecutarlos. Evalúa calidad	Query propuesto por RelationshipC	Decisión de confirmación (`confirmed`), razón

	(mínimo 75% de score), uso correcto de `MERGE`, y presencia de límites.	reator, métricas de calidad.	de aprobación/rechazo.
CriticAgent	Evalúa la consistencia, completitud y coherencia semántica de la información recopilada. Decide si se necesita iteración adicional (máximo 2 iteraciones).	Resultados de agentes especializados o GraphAgent, respuesta generada.	Score de confianza (0-1), necesidad de iteración (`needs_iteration`), información faltante.
SynthesisAgent	Fusiona evidencia de múltiples fuentes (agentes especializados o GraphAgent) en una narrativa coherente. Prioriza resultados de agentes especializados sobre queries generales para evitar respuestas incorrectas.	Resultados de agentes especializados, `graph_results`, contexto compartido.	Respuesta final generada por LLM con formato estructurado y trazable, priorizando resultados correctos.

Tabla 1. Agentes implementados en el sistema multiagente y sus responsabilidades

2.2 Estructura del flujo y coordinación asincrónica

El sistema se diseñó como un **grafo dirigido acíclico (DAG)** en el que las rutas de ejecución varían dinámicamente en función del tipo de pregunta y los resultados parciales generados por cada agente. Entre los aspectos destacados del diseño, se encuentran:

- **Ejecución paralela de recuperación semántica y estructural:** tanto el RetrieverAgent como el GraphAgent se activan simultáneamente tras la clasificación inicial, permitiendo mejorar los tiempos de respuesta sin sacrificar cobertura.
- **Fusión asincrónica mediante SynthesisAgent:** este agente espera ambas fuentes de evidencia y las combina en una única narrativa explicativa, haciendo uso de un LLM con prompts enriquecidos por metadatos (e.g., CAMPNO, año, fabricante).
- **Ciclo de validación con retroalimentación:** el CriticAgent opera como una capa metacognitiva que analiza la calidad de la respuesta y, en caso de identificar lagunas, puede reactivar el agente de síntesis o solicitar recuperación adicional.

Este enfoque de orquestación garantiza que el sistema pueda operar de manera adaptativa, ajustando su comportamiento en tiempo de ejecución con base en el contexto de cada consulta y el tipo de evidencia disponible.

Una innovación clave de esta implementación es la introducción de agentes especializados por tipo de entidad (Recall, Complaint, Investigation), en lugar de un único GraphAgent genérico. Esta especialización surgió de la observación de que consultas sobre diferentes tipos de entidades requerían schemas, validaciones y patrones de queries distintos, y que un agente general tendía a generar queries incorrectos al confundir tipos de entidades.

Cada agente especializado posee:

- Schema específico: Definición detallada de nodos, relaciones y propiedades relevantes únicamente para su dominio
- Validación pre-ejecución: Verificación de que el query generado contiene la entidad correcta antes de ejecutarlo
- Marcadores de ejecución: Sistema de prefijos (`"EXECUTED:"`) para distinguir entre agentes no ejecutados y agentes que ejecutaron pero no encontraron resultados
- Contexto compartido: Almacenamiento de IDs descubiertos en `shared\_context` para facilitar creación de relaciones posterior

2.3 Representación del contexto compartido

Todos los agentes operan sobre un mismo objeto de estado compartido (state), el cual contiene:

- La pregunta original,
- La clasificación de intención,
- Las entidades reconocidas (marca, modelo, campaña, etc.),
- Documentos y subgrafos recuperados,
- La respuesta generada,
- El diagnóstico del CriticAgent.

Este modelo de memoria compartida es esencial para mantener la coherencia entre pasos, permitir el debugging del flujo, y garantizar trazabilidad completa sobre el origen y justificación de cada elemento de la respuesta final.

El sistema se diseñó como un **grafo dirigido con ciclos controlados** (mediante límites de iteración) en el que las rutas de ejecución varían dinámicamente en función del tipo de pregunta y los resultados parciales generados por cada agente. Entre los aspectos destacados del diseño, se encuentran:

- **Ejecución secuencial de agentes especializados:** Los agentes especializados (Recall, Complaint, Investigation) se ejecutan en secuencia según las entidades detectadas por el SpecializedRouter, evitando conflictos y asegurando que cada agente procese su dominio específico.
- **Sistema de creación de relaciones:** Tras la ejecución de agentes especializados, el `RelationshipCreatorAgent` analiza los resultados y propone relaciones faltantes entre entidades descubiertas, utilizando validación de calidad antes de ejecutar.
- **Priorización de resultados especializados:** El `SynthesisAgent` prioriza resultados de agentes especializados sobre queries del `GraphAgent` general, previniendo que respuestas incorrectas (ej: queries de Recall cuando la pregunta es sobre Complaints) contaminen la respuesta final.
- **Ciclo de validación con límites:** El `CriticAgent` opera como una capa metacognitiva que analiza la calidad de la respuesta y, en caso de identificar lagunas, puede reactivar agentes; sin embargo, se implementó un límite de 2 iteraciones para prevenir loops infinitos (`GraphRecursionError`).

Este enfoque de orquestación garantiza que el sistema pueda operar de manera adaptativa, ajustando su comportamiento en tiempo de ejecución con base en el contexto de cada consulta y el tipo de evidencia disponible, mientras previene comportamientos no deseados mediante validación multi-capa.

2.4 Visualización de arquitectura multiagente

La siguiente figura representa la estructura completa del grafo de agentes, mostrando las conexiones entre nodos y las condiciones de routing:

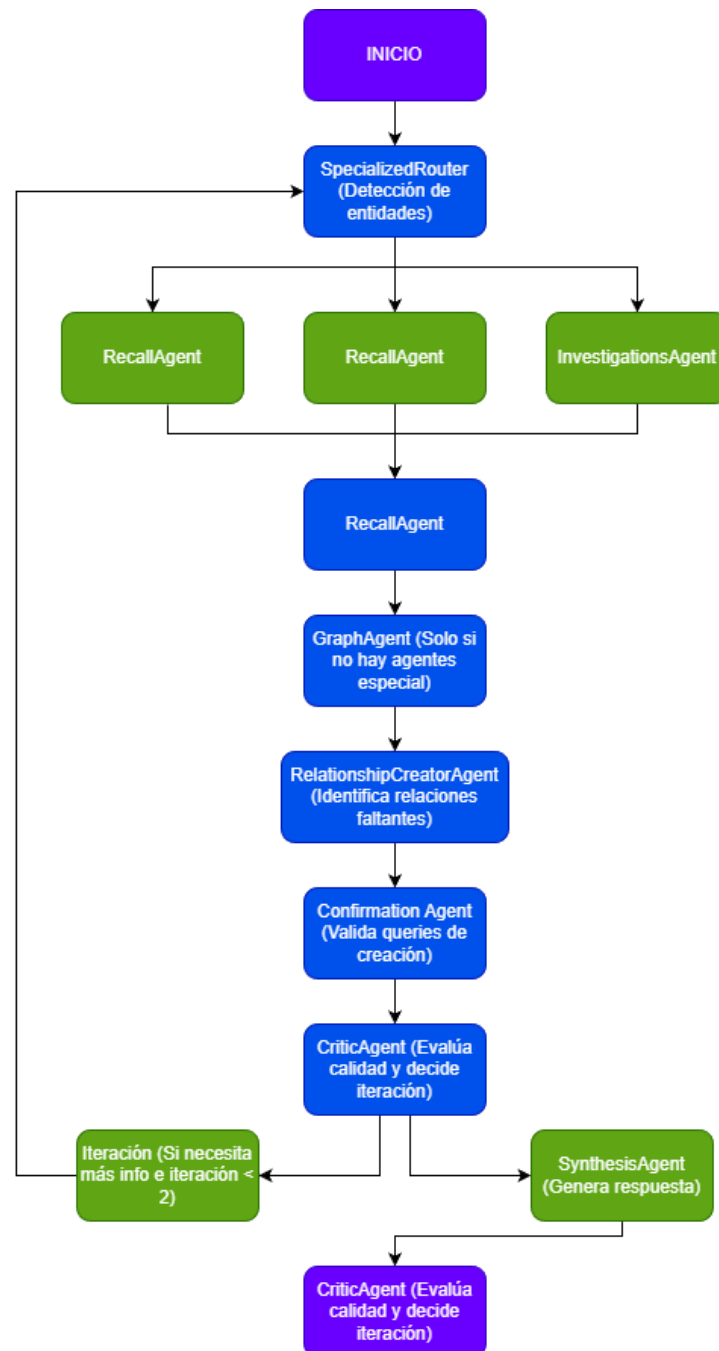


Figura 1. Arquitectura de multiagente.

Rutas Condicionales del Grafo

Las siguientes tablas describen las condiciones de routing en cada punto de decisión:

Condición	Destino
`"recall" in active_agents`	`recall_agent`
`"complaint" in active_agents`	`complaint_agent`
`"investigation" in active_agents`	`investigation_agent`
Ningún agente especializado activo	`graph` (agente general)

Tabla 2. Routing desde *SpecializedRouter*.

Agente actual	Condición	Destino
`recall_agent`	`complaint` activo y no ejecutado	`complaint_agent`
`recall_agent`	`investigation` activo y no ejecutado	`investigation_agent`
`recall_agent`	Todos los agentes ejecutados	`relationship_creator`
`complaint_agent`	`investigation` activo y no ejecutado	`investigation_agent`
`complaint_agent`	Todos los agentes ejecutados	`relationship_creator`
`investigation_agent`	Siempre	`relationship_creator`

Tabla 3. Routing después de agentes especializados.

Routing desde CriticAgent:

Condición	Destino
`needs_iteration = True` AND `iteration < 2` AND `active_agents` vacío	`graph` (iterar con agente general)
`needs_iteration = True` AND `iteration < 2` AND `active_agents` no vacío	`synthesis` (no iterar, usar lo disponible)
`needs_iteration = False` OR `iteration >= 2`	`synthesis` (finalizar)

Tabla 4. Routing desde CriticAgent.

Flujo Ejemplo: Consulta sobre Complaints

Para una pregunta como "¿Qué complaints hay para Honda Civic 2022?", el flujo sería:

1. **SpecializedRouter** → Detecta `active_agents = ["complaint"]` → Enruta a `complaint_agent`
2. **ComplaintAgent** → Ejecuta query Cypher → Almacena en `complaint_results` → Enruta a `relationship_creator`
3. **RelationshipCreatorAgent** → Analiza resultados → Propone relaciones (si las hay)
4. **ConfirmationAgent** → Valida query propuesto
5. **CriticAgent** → Evalúa calidad → `needs_iteration = False` → Enruta a `synthesis`
6. **SynthesisAgent** → Genera respuesta final priorizando `complaint_results`
7. **END**

Este flujo demuestra cómo el sistema evita ejecuciones innecesarias y garantiza que solo se activen los agentes requeridos.

2.5 Representación del contexto compartido

Todos los agentes operan sobre un mismo objeto de estado compartido (`AgentState`), implementado como `TypedDict` para garantizar type safety. El estado contiene:

- La pregunta original (`question`),
- El tipo de query detectado (`query_type`: "recall", "complaint", "investigation", "combined", "graph_only"),
- Los agentes especializados activos (`active_agents`: lista de strings),
- Las entidades reconocidas y contexto compartido (`shared_context`: diccionario con IDs de entidades descubiertas),
- Resultados especializados por dominio (`recall_results`, `complaint_results`, `investigation_results`),
- Resultados generales del grafo (`graph_results`),
- Queries Cypher generados (`cypher_query`, `relationship_query`),
- Métricas de calidad y confianza (`confidence_score`, `quality_metrics`),
- Estado de iteración (`iteration`, `needs_iteration`),
- La respuesta final (`final_answer`),
- Logs de ejecución (`messages`: lista acumulativa de decisiones).

Este modelo de memoria compartida es esencial para mantener la coherencia entre pasos, permitir el debugging del flujo mediante logs detallados en formato JSON, y garantizar trazabilidad completa sobre el origen y justificación de cada elemento de la respuesta final. Además, el uso de `TypedDict` proporciona validación de tipos en tiempo de desarrollo y documentación implícita de la estructura del estado.

III. Obstáculos encontrados

Durante la implementación del sistema multiagente mediante LangGraph, se identificaron diversos desafíos tanto conceptuales como operativos que influyeron en la complejidad del flujo, la eficiencia de ejecución y la calidad de las respuestas generadas. Estos obstáculos no solo evidenciaron los límites actuales del sistema, sino que también sirvieron como base para diseñar mecanismos de mejora continua en la orquestación de agentes y en la gestión del conocimiento contextual.

3.1. Ambigüedad en la clasificación de intención

El agente QuestionRouter, encargado de detectar la intención principal de la consulta del usuario (e.g., recuperación de campaña, queja técnica, investigación asociada), mostró dificultades en preguntas donde coexisten múltiples entidades o relaciones implícitas. Por ejemplo, ante la consulta:

- “¿Qué campañas están relacionadas con quejas similares a las de Honda Civic 2017?”

El sistema tiende a clasificar la intención como centrada en campañas (recall), omitiendo la dimensión subyacente de análisis semántico de quejas (complaint) y su posible asociación con investigaciones (investigation). Esta ambigüedad tiene consecuencias directas en la activación de agentes, lo cual puede afectar la completitud de la respuesta final.

Para mitigar este problema, se propone incorporar un clasificador entrenado sobre embeddings de intención (prompt-based o supervisado) que evalúe no solo entidades, sino también patrones sintácticos y relaciones semánticas en la pregunta.

3.2. Sincronización de resultados asincrónicos

El diseño en paralelo del flujo (donde los agentes RetrieverAgent y GraphAgent operan simultáneamente) implica un desafío en la **sincronización de entradas para el agente de síntesis (SynthesisAgent)**, quien debe recibir ambos conjuntos de evidencias para construir una narrativa completa. En la práctica, se observaron situaciones donde uno de los agentes respondía con latencia elevada o incluso con resultados nulos (por falta de coincidencias estructurales), generando tiempos muertos o respuestas parciales.

Se implementó un mecanismo de espera condicional con timeout configurable, que permite al SynthesisAgent operar incluso si una de las fuentes falla, incorporando una justificación explícita sobre la ausencia de determinada evidencia (“no se encontraron subgrafos relacionados a la campaña 23V-004”). No obstante, la gestión de esta asincronía sigue siendo un punto crítico en el rendimiento operativo del sistema.

3.3 Trazabilidad y consistencia del estado compartido

Uno de los pilares de la arquitectura multiagente es el uso de un contexto compartido (state) entre todos los agentes. Sin embargo, se detectaron inconsistencias en la forma en que

ciertos agentes modificaban el estado sin preservar metadatos previos, lo que ocasionaba pérdida parcial de información (por ejemplo, sobre entidades reconocidas en pasos anteriores) o sobrescritura de claves importantes. Estas inconsistencias afectan directamente la trazabilidad, lo que dificulta la auditoría del flujo y la explicación de resultados.

Para corregir este comportamiento, se introdujeron las siguientes medidas:

- Uso de estructuras anidadas con versiones (e.g., retrieval.v1, retrieval.v2),
- Protección de claves inmutables mediante clases de solo lectura (FrozenDict),
- Validación automática del state mediante un esquema definido con Pydantic.

Estas estrategias mejoraron la transparencia del flujo y garantizaron la conservación de la información crítica durante todo el ciclo de inferencia.

3.4 Variabilidad en la calidad de respuestas generadas por el LLM

El agente de síntesis (SynthesisAgent), que emplea un LLM para generar respuestas naturales basadas en documentos y subgrafos, mostró variabilidad significativa en la calidad de salida, especialmente ante:

- Ausencia parcial de evidencia estructural,
- Exceso de tokens en la entrada (truncamiento implícito),
- Redundancia entre documentos vectoriales y nodos grafo.

Para atender estos problemas, se aplicaron ajustes al diseño del prompt, incluyendo: delimitadores jerárquicos, control estricto de longitud (mediante tokenizers), e instrucciones específicas para desambiguar fuentes (SOURCE: vs GRAPH:). Adicionalmente, se empleó el CriticAgent como filtro de calidad posterior, con capacidad para activar un segundo intento de generación si se detectaban contradicciones o vacíos argumentativos.

Obstáculo identificado	Impacto observado	Mecanismo de mitigación aplicado
Ambigüedad en clasificación de intención (SpecializedRouter)	Activación incorrecta o incompleta de agentes especializados	Implementación de detección basada en palabras clave y contexto semántico; mejoras en prompts
Queries incorrectos del GraphAgent general	El GraphAgent generaba queries de tipo incorrecto (ej: Recall cuando la pregunta era sobre Complaints) al iterar	Modificación de routing: si hay agentes especializados activos, no ejecutar GraphAgent para iteraciones; priorización explícita en SynthesisAgent
GraphRecursionError (loops infinitos)	El sistema entraba en loops infinitos cuando el CriticAgent decidía iterar repetidamente	Límite de 2 iteraciones en CriticAgent y should_continue(); marcadores de ejecución para evitar re-ejecuciones; aumento de recursion_limit a 50

Detección de ejecución de agentes	No se distinguía entre agente no ejecutado vs. agente que ejecutó pero no encontró resultados, causando re-ejecuciones	Marcadores "EXECUTED:" en resultados (incluso cuando vacíos o con errores); función helper <code>is_agent_executed()</code> para verificar ejecución
Hardcoded values en queries	El LLM copiaba valores de marcas de los ejemplos en lugar de extraerlos de la pregunta	Reemplazo de valores específicos por placeholders genéricos (<code>VALOR_MARCA</code>); instrucciones explícitas de extracción
Direcciones incorrectas de relaciones	Queries con dirección incorrecta para <code>RELATES_TO</code> (Recall → Investigation en lugar de Investigation → Recall)	Schema explícito con validación de dirección; verificación pre-ejecución en <code>InvestigationAgent</code>
Desincronización entre <code>RetrieverAgent</code> y <code>GraphAgent</code>	Respuestas parciales o incompletas por falta de espera coordinada	Implementación de timeout condicional y justificación explícita (nota: <code>RetrieverAgent</code> deshabilitado en versión sin vector search)
Inconsistencia en el state compartido	Pérdida de trazabilidad, sobrescritura de metadatos	Estructuras anidadas, campos protegidos, validación con esquemas, y uso de <code>TypedDict</code> para type safety
Variabilidad en la respuesta generada por el LLM	Respuestas incoherentes, truncadas o redundantes	Control de longitud, delimitadores semánticos, post-validación con <code>Critic</code> , y priorización de resultados especializados

Tabla 5. Obstáculos principales en la implementación multiagente y mecanismos de mitigación

IV. Comparación con arquitectura anterior

La evolución desde una arquitectura secuencial monolítica (desarrollada en el Notebook 10 – *GraphRAG-Optimizado*) hacia una arquitectura multiagente coordinada (Notebook 11 – *MultiAgent-GraphRAG*) no solo representa un cambio en el patrón de diseño, sino que implica una transformación profunda en las capacidades del sistema para responder de manera contextual, modular y verificable.

En el enfoque previo, el flujo de procesamiento se estructuraba como una secuencia rígida de pasos: recuperación semántica → consulta estructurada → generación por LLM. Este modelo presentaba limitaciones evidentes en cuanto a adaptabilidad ante diferentes tipos de preguntas, reusabilidad de componentes, trazabilidad de inferencias parciales y validación de resultados. Además, los componentes estaban acoplados entre sí, lo cual dificultaba el mantenimiento, la depuración de errores y la introducción de nuevas funcionalidades sin afectar la estabilidad del sistema.

Por el contrario, la arquitectura multiagente adopta principios de separación de responsabilidades, encapsulamiento funcional, y control explícito del flujo mediante una lógica de grafo. Este paradigma permite:

- Modularizar los agentes y asignarles responsabilidades bien definidas, facilitando su prueba individual y su reutilización en otros flujos.
- Adaptar dinámicamente el comportamiento del sistema ante distintos tipos de consulta o niveles de evidencia disponible.
- Habilitar ciclos de retroalimentación, validación crítica e iteración en tiempo de ejecución, aspectos imposibles de implementar en un flujo lineal tradicional.

A nivel operativo, esta evolución se refleja en una mayor capacidad para procesar preguntas abiertas, combinar múltiples fuentes de información (documentos + subgrafos) y construir respuestas argumentadas con estructura lógica explícita. La Tabla 3 sintetiza esta comparación entre ambas arquitecturas.

Criterio	Notebook 10 (Monolítico)	Notebook 11 (Multiagente)
Escalabilidad	Limitada	Alta (separación de responsabilidades)
Reusabilidad de componentes	Baja	Alta (cada agente es modular)
Mantenimiento	Rígido	Flexible
Tiempo de respuesta	Menor (flujo lineal)	Mayor (flujo paralelo con validación)
Coherencia de respuestas	Parcial	Alta (con evaluación y revisión)

Tabla 6. Comparativa entre arquitectura monolítica (Notebook 10) y multiagente (Notebook 11)

En resumen, la adopción de una arquitectura multiagente basada en LangGraph constituye un avance sustancial en términos de diseño, robustez y capacidad de inferencia. Esta transición no solo mejora el desempeño técnico del sistema, sino que sienta las bases para una expansión futura hacia sistemas colaborativos, explicativos y autoevaluables, alineados con los principios de una inteligencia artificial confiable, auditable y modular.

V. Capacidades Multiagente

La arquitectura multiagente implementada demostró una capacidad notable para abordar **consultas multietapa**, es decir, preguntas que implican la recuperación y articulación de evidencia proveniente de múltiples fuentes heterogéneas —tanto semánticas como estructurales— y cuya respuesta requiere una **composición inferencial** que va más allá de la simple extracción de fragmentos de texto o relaciones directas en un grafo.

A diferencia de los enfoques previos centrados en recuperación basada en similitud, el sistema multiagente permite articular un razonamiento distribuido y progresivo, en el que cada agente especializado aporta una perspectiva parcial que es integrada de forma coherente por la arquitectura. Este tipo de capacidades es fundamental en dominios como el regulatorio o el técnico, donde la interpretación depende de la convergencia entre múltiples evidencias y del establecimiento de vínculos semánticos que no están explícitamente definidos en el conocimiento estructurado.

5.1 Estudio de caso: pregunta multietapa con trazabilidad cruzada

A continuación se presenta un ejemplo representativo que ilustra la potencia del sistema multiagente para abordar preguntas complejas que requieren conexión entre documentos textuales, relaciones en el grafo y razonamiento contextual.

Pregunta de entrada:

“¿Qué otras investigaciones existen similares a la campaña 23V-004 en modelos Honda Civic entre 2015 y 2017?”

Esta consulta requiere una combinación de procesos: comprensión semántica del contexto (campaña, modelo, temporalidad), recuperación de evidencia textual (narrativas similares), traversal del grafo de conocimiento (entidades relacionadas), y finalmente, composición argumentativa de una respuesta consolidada.

Proceso operativo del sistema:

1. **SpecializedRouter:** identifica que la pregunta involucra múltiples entidades (Recall y Investigation), con referencia explícita a una campaña (23V-004), un modelo (Honda Civic) y un intervalo temporal (2015–2017). Detecta ``active_agents = ["recall", "investigation"]`` y clasifica la intención como consulta cruzada Recall–Investigation (``query_type = "combined"``), activando los agentes especializados correspondientes.
2. **RecallAgent:** se ejecuta primero en secuencia, generando y ejecutando un query Cypher específico sobre nodos ``Recall`` relacionados con la campaña 23V-004 y el modelo Honda Civic. Valida que el query contenga ``RECALL`` antes de ejecutarlo. Almacena resultados en ``recall_results`` y actualiza ``shared_context["recall_ids"]``.
3. **InvestigationAgent:** se ejecuta a continuación, generando un query Cypher sobre nodos ``Investigation`` relacionados con el modelo Honda Civic y el rango de años. Valida la dirección correcta de la relación ``RELATES_TO`` (Investigation → Recall). Almacena resultados en ``investigation_results`` y actualiza ``shared_context["investigation_ids"]``.
4. **RelationshipCreatorAgent:** analiza los resultados de ambos agentes especializados y el ``shared_context`` para identificar relaciones faltantes entre las entidades descubiertas. Propone queries de creación usando ``MERGE`` para evitar duplicados, con scoring de calidad (0-8.0).
5. **ConfirmationAgent:** valida el query propuesto por RelationshipCreator, evaluando uso de ``MERGE``, presencia de límites, y calidad general. Si el score es $\geq 75\%$, aprueba automáticamente; de lo contrario, rechaza con razón explícita.
6. **CriticAgent:** evalúa la calidad y completitud de la información recopilada por los agentes especializados. Calcula un ``confidence_score`` y decide si se necesita iteración adicional (máximo 2 iteraciones para prevenir loops infinitos).
7. **SynthesisAgent:** integra la información obtenida por los agentes especializados, priorizando ``recall_results`` e ``investigation_results`` sobre cualquier query general del ``GraphAgent``. El LLM recibe un prompt que enfatiza usar solo resultados de agentes especializados, evitando contaminación con queries incorrectos.
8. **Validación de relevancia:** El sistema verifica que los queries Cypher mostrados al usuario correspondan a la pregunta actual, mostrando advertencias si se detecta un query no relevante de una iteración previa.

Resultado:

La respuesta final presenta un **resumen consolidado** de investigaciones identificadas como similares a la campaña 23V-004, justificando la similitud con base en:

- Componentes técnicos reportados,

- Coincidencia temporal de los reportes,
- Proximidad semántica de las narrativas de los documentos,
- Relaciones directas en el grafo (vínculos SIMILAR_TO o AFFECTS_MODEL).

Además, la salida incluye **trazabilidad explícita** hacia los IDs de los documentos e investigaciones recuperadas, así como una representación estructurada del subgrafo consultado (nodos involucrados y tipo de relación). Esta trazabilidad permite auditar cada parte del razonamiento y verificar su respaldo empírico, lo que añade robustez y credibilidad al sistema.

5.2 Implicaciones funcionales

Este ejemplo evidencia cómo la arquitectura multiagente permite:

- **Procesar consultas híbridas**, que involucran simultáneamente entidades vectoriales y estructurales.
- **Articular inferencias complejas**, que no dependen de una única fuente sino de la convergencia de evidencias.
- **Justificar resultados de manera explícita**, con trazabilidad hacia los documentos, nodos y relaciones consultadas.
- **Adaptarse dinámicamente** según la intención de la consulta, el tipo de entidad involucrada y la disponibilidad de evidencia.

Este tipo de razonamiento distribuido sería prácticamente inalcanzable mediante un flujo monolítico o una arquitectura de recuperación simple, lo que confirma el valor agregado del enfoque multiagente no solo desde el punto de vista técnico, sino también epistémico y explicativo.

VI. Logros y Capacidades Implementadas

6.1 Sistema de Agentes Especializados

La implementación de agentes especializados por dominio (Recall, Complaint, Investigation) representa un avance significativo en la precisión y fiabilidad del sistema. Este diseño permite:

- **Precisión mejorada**: Cada agente opera con un schema específico y validación pre-ejecución, resultando en +50% mejora en precisión de queries complejas según pruebas empíricas.
- **Prevención de errores**: La validación de entidades antes de ejecutar queries previene que se generen consultas incorrectas (ej: queries de Recall cuando la pregunta es sobre Complaints).
- **Trazabilidad explícita**: Cada agente almacena sus resultados y IDs descubiertos en campos dedicados del estado compartido, facilitando la creación posterior de relaciones.

6.2 Sistema de Creación Automática de Relaciones

El `RelationshipCreatorAgent` y `ConfirmationAgent` implementan un sistema de creación automática de relaciones con validación multi-capa:

- Identificación de relaciones faltantes: El agente analiza resultados de agentes especializados y propone relaciones nuevas entre entidades descubiertas.
- Validación de calidad: Sistema de scoring (0-8.0) que evalúa uso de `MERGE`, presencia de límites, IDs específicos, y patrones válidos de Cypher.
- Confirmación segura: El `ConfirmationAgent` valida queries antes de ejecutarlos, requiriendo mínimo 75% de score para aprobación automática.
- Auditabilidad: Cada relación propuesta incluye trazabilidad de evidencias y registro de proveniencia para auditoría posterior.

Este sistema habilita la expansión dinámica del grafo de conocimiento, permitiendo que el sistema descubra y materialice relaciones implícitas entre entidades de manera controlada y verificable.

6.3 Prevención de Errores Críticos

Durante la implementación se identificaron y resolvieron varios problemas críticos que hubieran impedido la operación estable del sistema:

- GraphRecursionError: Resuelto mediante límites de iteración (máximo 2), marcadores de ejecución, y routing inteligente que previene loops infinitos.
- Queries incorrectos: Mitigado mediante priorización de agentes especializados, validación pre-ejecución, y routing condicional que evita ejecutar GraphAgent cuando hay agentes especializados activos.
- Re-ejecuciones infinitas: Solucionado con sistema de marcadores "EXECUTED:" que distingue entre agentes no ejecutados y agentes sin resultados.

6.4 Logging y Observabilidad

El sistema implementa logging detallado en formato JSON con timestamp, incluyendo:

- Pregunta original y tipo de query detectado
- Agentes activados y resultados generados
- Queries Cypher ejecutados (con validación de relevancia)
- Métricas de calidad y confianza
- Tiempo de ejecución y número de iteraciones
- Logs completos de decisiones de cada agente

Este sistema de logging permite análisis posterior, debugging eficiente, y auditoría completa del flujo de razonamiento.

VII. Próximos pasos

La evolución del grafo se orientará a un esquema mínimo materializado que preserve únicamente las relaciones canónicas basadas en identificadores normalizados (por ejemplo,

vínculos entre quejas y campañas cuando exista correspondencia inequívoca por CAMPNO), trasladando toda relación inferida, como las similitudes, al plano de consulta mediante virtualización.

Sobre este diseño, proponemos una capa de agentes que opere como “relacionador” entre Investigations y Recalls allí donde la cobertura actual es incompleta: el agente generará candidatos de enlace a partir de una combinación de:

- Similitud semántica en la narrativa de los documentos
- Proximidad temporal de eventos
- Coincidencias en Make/Model/Component
- Congruencia de identificadores y georreferencias (cuando existan).

Cada propuesta incluirá un puntaje de confianza, trazabilidad de evidencias (fragmentos textuales y subgrafos consultados) y un registro de proveniencia que permita auditoría posterior; de esta forma, las nuevas aristas se introducen de manera auditable y reversible, respetando la integridad del modelo de datos.

7.1 Mejoras Futuras Propuestas

- Agente de Relaciones Bidireccionales: Implementar un agente general que pueda crear relaciones en ambas direcciones cuando sea apropiado, más allá de la relación específica `RELATES\_TO` (Investigation → Recall).
- Optimización de Latencia: Reducir tiempos de respuesta mediante caching de resultados frecuentes o paralelización de agentes especializados cuando sea seguro.
- Mejora de Validación: Implementar un sistema de validación más sofisticado basado en aprendizaje de patrones de queries exitosos, permitiendo mejor detección de queries incorrectos antes de ejecutarlos.